

PhishTrainer — Features

Frontend Systems (SS 2026) — Milestone 5

PhishTrainer is a React application that trains users to recognise phishing emails through realistic simulated scenarios and immediate, personalised feedback. This milestone completes the core feature set and connects all views into a coherent training flow.

Implemented Functionality

- **Dashboard:** accuracy meter, level and XP progress, a daily streak, earned badges, and a difficulty selector (Easy / Medium / Hard) with a recommended option. The chosen difficulty is stored on the backend and starts the training flow.
- **Scenario:** renders each email with sender, subject and body. Suspicious phrases in the text are clickable tokens that the user marks as red flags before choosing a final verdict (phishing or legitimate). Scenarios are grouped by difficulty and played in sequence.
- **Feedback:** computes the result from the user's actual verdict and marked flags — correct/incorrect, XP earned, and how many red flags were found — explains each flag, and saves the outcome so it updates the dashboard statistics and appears in the history.
- **History:** lists past sessions with summary statistics, a controlled search field, and combinable filters for result and difficulty.

Major Technical Decisions

- **Shared data hook:** a custom `useApiData` hook centralises every asynchronous fetch together with its loading and error state, so all four views handle backend communication the same way (DRY).
- **Query-parameter navigation:** the Scenario view passes the selected verdict and marked flags to the Feedback view through URL parameters, so feedback is derived from the user's real choices instead of a fixed record and navigation stays stateless and shareable.
- **Mock backend:** a JSON Server instance provides all data over REST. The app reads with GET, creates new history entries with POST, and updates the dashboard with PATCH, so finished scenarios are persisted — demonstrating two-way asynchronous communication.
- **Reusable components and semantics:** shared components (`StatCard`, `SelectableButtons`, `EmptyState`, `ResultBanner`, `HistoryEntry`) and a `services` layer keep the code maintainable, and each view is rendered as a semantic `<main>` landmark.
- **Responsive styling:** CSS custom properties act as design tokens and a single breakpoint adapts the grid layouts for small screens.

Remaining Limitations

- The application uses a single mock user without authentication, and the persisted progress is shared globally rather than per account.
- Red-flag detection relies on an exact text match between the stored flag and the email content, which constrains how new scenarios are authored.

Acknowledging AI Technologies

This document and parts of the implementation were revised with the support of Claude (Anthropic). The tool was used to improve structure, refactor towards shared components, and polish wording. All project-specific content, technical decisions, and code were reviewed and understood by the team.